



ROBOT SIMULATION AND HARDWARE PROJECT REPORT

MECH 472
Mechatronics and Automation
Presented to
Dr. Brandon W. Gordon

By
Jemima Hossain [40208886]
Elale Issayas Herouy [40156635]
Timothée Brunet [40197228]
Mohammad Umer [40150744]

Concordia University, Montreal
May 5th, 2025

WORK DISTRIBUTION

Jemima Hossain: Defense program

- defence()
- hide_behind_obstacle()
- escape_opponent()
 - Zigzag path
- Tried but had difficulty with hardware

Elale Issayas Herouy: A* Pathfinding algorithm and Hardware implementation

-Image processing tests for different threshold values were made and optimum values for robot centroid detection were selected.

-For the attack method, the first trial was developing an A* pathfinding algorithm where a Matrix with a size the same as number of pixels was used, where the g-cost and h-cost was updated for the nearest pixels and the algorithm selects the pixel with the lowest h-cost. Leading to the shortest path to the target but it was later removed as raycasts offered a better and simpler alternative.

Timothée Brunet: General functions, helped with defense and attack

- **opp_keyboard_input()**
- **detect_color()**
- **vision_process()**
- Attack:
 - Added raycast visualizers to **forward_raycast_obstacle()** and **forward_raycast_boundary()**
 - Adjusted corners positions and raycast direction in **calculate_robot_corners()** and **move_in_clear_direction()** to better cover the robot
 -
- Defense:
 - Adjusted **hide_x** and **hide_y** in **hide_behind_obstacle()** such that the robot does not target a point too close to the border
 - Added point marker that shows **hide_x** and **hide_y** on screen for debugging
 - Made the robot move perpendicularly to the opponent under **escape_opponent()** instead of away, making it harder to hit and increasing the chances of hiding again
 - Implemented the **choose_obstacle()** function for the robot to choose the “best” obstacle to hide behind at the start of the run
- General debugging

Mohammad Umer: Attack program, hardware implementation

- *Attack 1st iteration:*

- Robot finds nearest obstacle, moves within 60-80 pixels of it and tries to move tangentially to it while obtaining line of sight with enemy.

First implementation of following functions:

```
bool check_boundaries();  
void free_movement();  
void moving_process();  
bool check_line_of_sight();  
bool is_near_boundary();  
bool turn_from_boundary();  
bool check_boundary_reset_stage();  
void move_tangential()  
ObstacleInfo find_nearest_obstacle()  
void align_with_obstacle()  
void approach_obstacle()
```

A bunch of these functions were improved and reused for the defense strategy instead.

- *Attack 2nd iteration:*

First implementation of following functions:

```
void visualize_collision_detection();  
bool rotation_check();  
bool is_robot_stuck();
```

- Implemented `int return_obstacles();`
Returns obstacle centroid location and label based on the assumption that since the obstacle are the largest objects; the mass associated with their label is also the highest.
- Hardware side implemented and verified serial communication over bluetooth.

GENERAL FUNCTIONS

Manual opponent (using simulation mode 0):

For texting, we used a manual opponent controlled with the function: `opp_keyboard_input()`.

```
int opp_keyboard_input(int &opp_l, int &opp_r) {
    if (KEY(VK_LEFT) && KEY(VK_UP)) {
        opp_l = 1500;
        opp_r = 2000;
    }
    else if (KEY(VK_RIGHT) && KEY(VK_UP)) {
        opp_l = 1000;
        opp_r = 1500;
    }
    else if (KEY(VK_UP)) {
        opp_l = 1000;
        opp_r = 2000;
    }
    else if (KEY(VK_DOWN)) {
        opp_l = 2000;
        opp_r = 1000;
    }
    else if (KEY(VK_LEFT)) {
        opp_l = 2000;
        opp_r = 2000;
    }
    else if (KEY(VK_RIGHT)) {
        opp_l = 1000;
        opp_r = 1000;
    }
    else {
        opp_l = 1500;
        opp_r = 1500;
    }
    return 0;
}
```

Acquiring robot and obstacle positions:

We modified the `centroid()` function to return the mass of each object.

We then used the obtained masses in a modified version of `track_object()` function to separate robot markers objects from the rest by size using a double threshold.

```
void detect_color(int nlabel) {
    1byte* p;
    int Rc, Gc, Bc;
    p = rgb1.pdata + 3*((int)ic[nlabel] + (int)jc[nlabel] * IMAGE_WIDTH);
    Bc = *p;
    Gc = *(p + 1);
    Rc = *(p + 2);

    //cout << Rc << " " << Gc << " " << Bc << " at i = " << (int)ic[nlabel] << ", j = " << (int)jc[nlabel] << endl;

    if ((Rc > 200) && (Gc < 95) && (Bc < 80)) {
        // detect red marker
        //cout << "Found red marker at i = " << (int)ic[nlabel] << ", j = " << (int)jc[nlabel] << endl;
        draw_point_rgb(rgb1, (int)ic[nlabel], (int)jc[nlabel], 100, 0, 0);
        #2pos[2] = (double)ic[nlabel];
        #2pos[3] = (double)jc[nlabel];
    }

    else if ((Rc < 75) && (Gc > 190) && (Bc < 135)) {
        // detect green marker
        //cout << "Found green marker at i = " << (int)ic[nlabel] << ", j = " << (int)jc[nlabel] << endl;
        draw_point_rgb(rgb1, (int)ic[nlabel], (int)jc[nlabel], 0, 255, 0);
        #2pos[0] = (double)ic[nlabel];
        #2pos[1] = (double)jc[nlabel];
    }

    else if ((Rc < 55) && (Gc < 165) && (Bc > 200)) {
        // detect blue marker
        //cout << "Found blue marker at i = " << (int)ic[nlabel] << ", j = " << (int)jc[nlabel] << endl;
        draw_point_rgb(rgb1, (int)ic[nlabel], (int)jc[nlabel], 0, 0, 255);
        #2pos[2] = (double)ic[nlabel];
        #2pos[3] = (double)jc[nlabel];
    }

    else if ((Rc > 220) && (Gc > 190) && (Bc < 130)) {
        // detect orange marker
        //cout << "Found orange marker at i = " << (int)ic[nlabel] << ", j = " << (int)jc[nlabel] << endl;
        draw_point_rgb(rgb1, (int)ic[nlabel], (int)jc[nlabel], 255, 255, 0);
        #2pos[0] = (double)ic[nlabel];
        #2pos[1] = (double)jc[nlabel];
    }

    //else { cout << Rc << " " << Gc << " " << Bc << " at i = " << (int)ic[nlabel] << ", j = " << (int)jc[nlabel] << endl;
}
```

After that, we used the `detect_color()` function, to differentiate the markers, find their position and mark them on screen for debugging.

Marker positions are written in `R1pos[4]` and `R2pos[4]` arrays.

ATTACK

Robot A

For our attack program, the robot wanders around, avoiding obstacles and image borders, until it obtains line-of-sight with the opponent. Once that is achieved, it moves towards the robot and turns towards it. Once our robot is aligned, it fires the laser and captures an image of the result. To check the distance to the obstacles and whether line-of-sight is achieved, we use a number of ray casts with different properties. These ray casts are visualized on the screen in different colors thanks to the `draw_point_rgb()` function. We also implemented markers at each corner of the robot that change color (green to red), when the robot is too close to an obstruction. The raycasts are like having 4 ultrasonic sensors at the 4 corners of the robot.

Important functions are called via `tracking_process()`, `vision_process()` and `moving_process()` in the various `program.cpp` files in the main of the program.

Robot B

For robot B, we simply changed the mode in lines:

```
mode = 1;  
set_simulation_mode(mode);
```

From 1 to 2, and switched around the marker colors such that our robot is orange and blue instead of the green and red. This is done by simply swapping the `R1pos[]` and `R2pos[]` arrays in the `detect_color` function.

DEFENSE

```
1291 void defense(int& pw_l, int& pw_r) {
1292     bool clear_sight = check_line_of_sight(label);
1293
1294     //if an obstacle is detected
1295     if (!clear_sight) {
1296         cout << "Obstacle detected Attempting to hide.\n";
1297         hide_behind_obstacle(pw_l, pw_r);
1298     }
1299     //if there is no obstacle
1300     else {
1301         cout << "Exposed. Escaping now.\n";
1302         escape_opponent(pw_l, pw_r);
1303     }
1304 }
```

The defense strategy (**defense() function**) has two modes:

- 1) An obstacle is blocking the robot from the opponent. The robot shields itself from the opponent by hiding behind that obstacle. This is the `hide_behind_obstacle()` function.
- 2) There is no obstacle protecting the robot. The robot moves away from the opponent in the opposite direction. This is the `escape_opponent()` function.

The robot also is able to choose the closest obstacle to hide behind through the `choose_obstacle()` function. This is more of a safeguard function since the main strategy is to use the obstacle on the path between the robot and the opponent.

1) `hide_behind_obstacle()` function:

Line 1348 to line 1351:

- We begin by locating the point that is directly behind the object (from the opponent's perspective)
- And we calculate the point (`hide_x`, `hide_y`) to which we want to move
 - We calculate for the distance between the obstacle and the opponent
 - We extend that vector by the same distance to reach the far side of the obstacle (from the opponent's perspective)

Line 1352 to end:

- The rest of the function is comparable to the `target_opponent()` function.
- The difference is that the robot simply rotates and makes a movement , it does not look to face the opponent and shoot a laser since it is in defense mode

2) `escape_opponent()` function:

- The program first locates the opponent, calculates its angle relative to the robot's angle, and the robot escapes at 180 degree from it.

- When escaping away, the robot moves in a zigzag path. This makes it difficult for the opponent to aim and follow:
 - It looks jittery but it is because the length of the displacement should be adjusted

```

1351 //Zigzag movement
1352 zigzag_counter++;
1353 if (zigzag_counter >= ninterval) {
1354     if (zigzag_dir == 1) {
1355         zigzag_dir = 2;
1356     }
1357     else {
1358         zigzag_dir = 1;
1359     }
1360     zigzag_counter = 0;
1361 }
1362
1363 //if we are in range. just move forward
1364 if (fabs(angle_diff) < ANGLE_THRESHOLD) {
1365     //zigzag movement
1366     if (zigzag_dir == 1) {
1367         pw_l = STOP - FORWARD_SPEED + TURN_SPEED;
1368         pw_r = STOP + FORWARD_SPEED;
1369     }
1370     else if (zigzag_dir == 2) {
1371         pw_l = STOP - FORWARD_SPEED;
1372         pw_r = STOP + FORWARD_SPEED + TURN_SPEED;
1373     }
1374
1375     //if we are pointed left of the opponent, turn left
1376 }
1377 else if (angle_diff > 0) {
1378     pw_l = STOP + FORWARD_SPEED;
1379     pw_r = STOP + FORWARD_SPEED;
1380     //if we are pointed right of the opponent, rotate right
1381 }
1382 else {
1383     pw_l = STOP - FORWARD_SPEED;
1384     pw_r = STOP - FORWARD_SPEED;
1385 }
1386
1387 }

```

In the main function for attack mode, lines 257 to 260 are replaced by

```

257 else if (rotation_check()) {
258     defense(pw_l, pw_r);
259 }

```

// ***DEFENSE MODE***

3) choose_obstacle () function:

To determine which obstacle, the robot will try to hide behind, we used the following function:

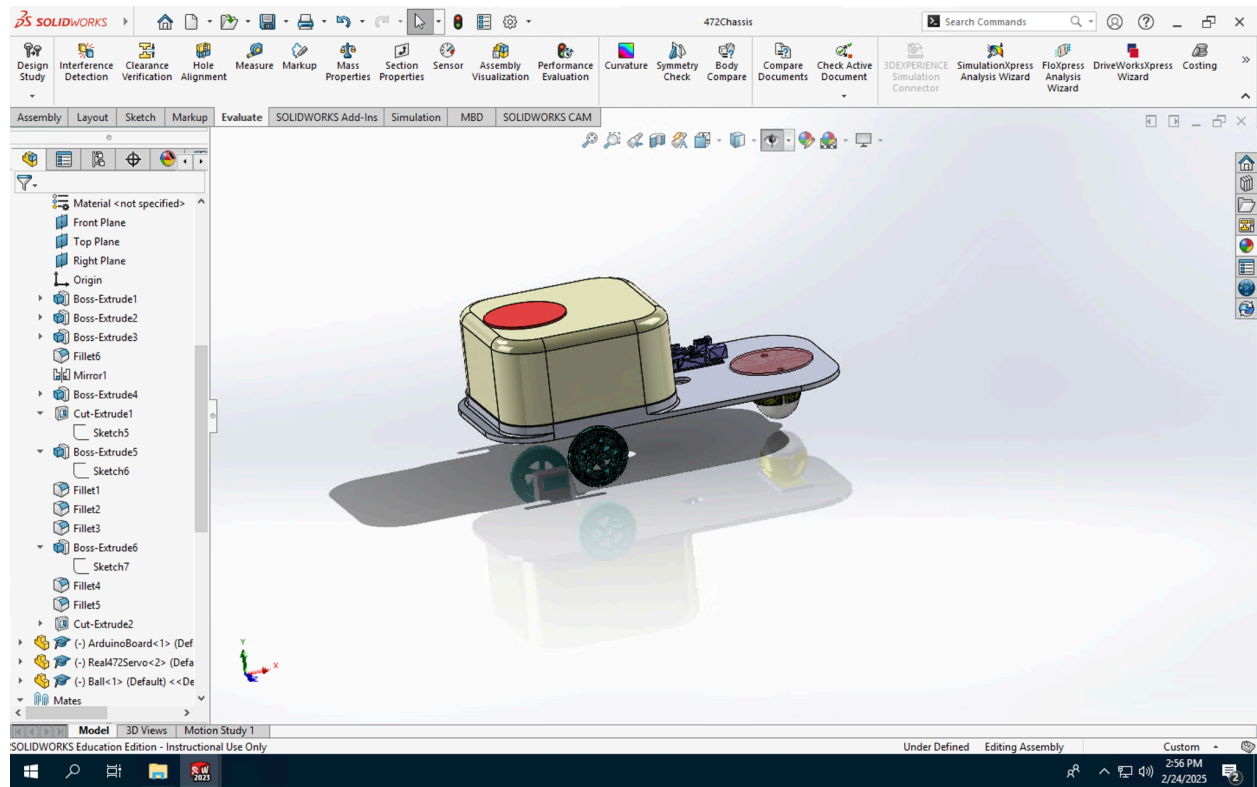

```

int choose_obstacle() {
    double maxdist = 0, dist, minDist = 150;
    int chosen_obstacle = 0;
    for (int i = 0; i < N_obs; i++) {
        bool borderdist = true;
        // checks if obstacle is too close to border to be hidden behind
        if (Opos[2 * i] > 640 - minDist || Opos[2 * i] < 0 + minDist
            || Opos[2 * i + 1] > 480 - minDist || Opos[2 * i + 1] < 0 + minDist) borderdist = false;
        if (borderdist) {
            //dist = abs(sqrt(pow(Rlpos[0] - Opos[2 * i], 2.0) + pow(Rlpos[1] - Opos[2 * i + 1], 2.0)));
            dist = abs((Rlpos[0] - Opos[2 * i]) + (Rlpos[1] - Opos[2 * i + 1])); // good enough approx
            //approximation of distance between obstacle and robot
            if (maxdist < dist) {
                maxdist = dist;
                chosen_obstacle = i;
            }
        }
    }
    cout << "Best obstacle determined as obstacle nb " << chosen_obstacle << endl;
    return chosen_obstacle;
}

```

The choose_obstacle() function is called once at the start of the program, and selects the closest obstacle to our robot (approximately) that is far enough from the border to allow the robot to move around to hide from the opponent.

HARDWARE



The hardware chassis is composed of a cardboard base, with bolted 3D printed parts to hold the motors in front, and the ping pong ball at the back.



Before while loop:

```
//  
HANDLE hSerial;  
LPCSTR portName = "\\\\.\\COM6"; // Replace with your HC-05 COM port  
  
hSerial = CreateFileA(portName,  
    GENERIC_WRITE,  
    0,  
    NULL,  
    OPEN_EXISTING,  
    0,  
    NULL);  
  
if (hSerial == INVALID_HANDLE_VALUE) {  
    std::cerr << "Error opening serial port\\n";  
    return 1;  
}  
  
// Configure serial port parameters  
DCB dcbSerialParams = { 0 };  
dcbSerialParams.DCBlength = sizeof(dcbSerialParams);  
  
if (!GetCommState(hSerial, &dcbSerialParams)) {  
    std::cerr << "Error getting serial parameters\\n";  
    CloseHandle(hSerial);  
    return 1;  
}  
  
dcbSerialParams.BaudRate = CBR_9600;  
dcbSerialParams.ByteSize = 8;  
dcbSerialParams.StopBits = ONESTOPBIT;  
dcbSerialParams.Parity = NOPARITY;  
  
if (!SetCommState(hSerial, &dcbSerialParams)) {  
    std::cerr << "Error setting serial parameters\\n";  
    CloseHandle(hSerial);  
    return 1;  
}  
  
// Allow Bluetooth to initialize  
Sleep(2000);  
//  
while(1) {
```

Inside while loop:

```
// Reflect PWM  
pw_l_m = 1500 - (pw_l - 1500);  
pw_r_m = 1500 - (pw_r - 1500);  
  
// Build serial command with laser control  
std::string command = std::to_string(pw_l_m) + "," +  
    std::to_string(pw_r_m) + "," +  
    std::to_string(laser) + "\\n";  
  
DWORD bytesWritten;  
BOOL status = WriteFile(hSerial, command.c_str(), command.length(), &bytesWritten, NULL);  
if (!status) {  
    std::cerr << "Write failed, exiting...\\n";  
    break;  
}  
else {  
    std::cout << "Sent: " << command;  
}
```

The message is communicated in the form 1500,1500,1 and parsed as pw_l, pw_r and laser values. The communication frequency is capped to 50Hz.

Arduino sketch:

```
#include <Servo.h>

Servo servo1, servo2;

int pwm1 = 1500;
int pwm2 = 1500;
bool servosActive = false;
const int laserPin = 8;

void setup() {
  Serial.begin(9600);
  pinMode(laserPin, OUTPUT);
  digitalWrite(laserPin, LOW); // Laser OFF at startup
}

void loop() {
  if (Serial.available()) {
    String input = Serial.readStringUntil('\n'); // Format: "1500,1500,1"

    int comma1 = input.indexOf(',');
    int comma2 = input.indexOf(',', comma1 + 1);

    if (comma1 > 0 && comma2 > comma1) {
      pwm1 = input.substring(0, comma1).toInt();
      pwm2 = input.substring(comma1 + 1, comma2).toInt();
      int laserVal = input.substring(comma2 + 1).toInt();

      // Apply PWM directly
      if (!servosActive) {
        servo1.attach(9);
        servo2.attach(10);
        servosActive = true;
      }

      servo1.writeMicroseconds(pwm1);
      servo2.writeMicroseconds(pwm2);

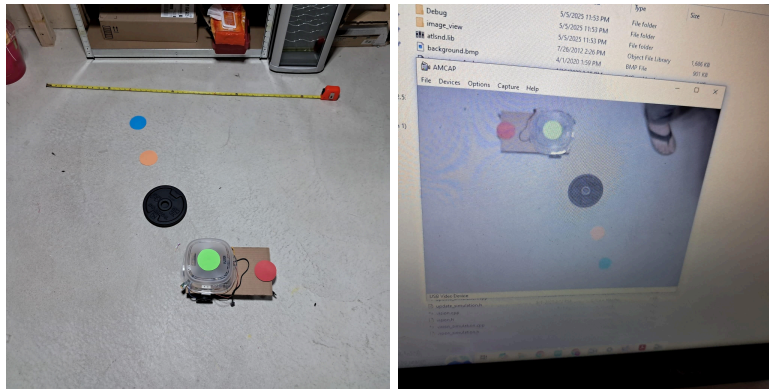
      // Laser control
      digitalWrite(laserPin, laserVal == 1 ? HIGH : LOW);
    }
  }

  if (servosActive) {
    // Maintain last command
    servo1.writeMicroseconds(pwm1);
    servo2.writeMicroseconds(pwm2);
  }

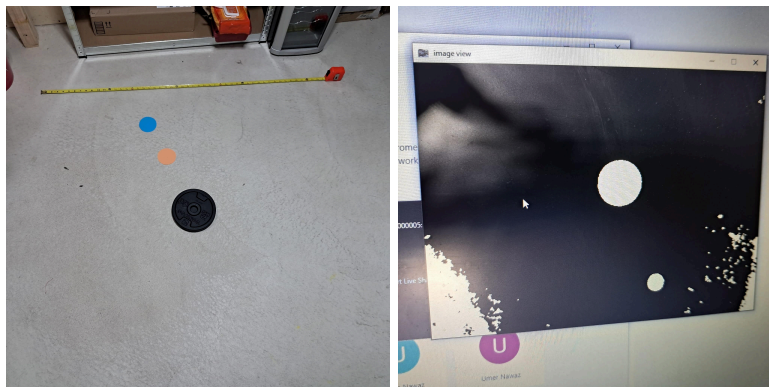
  delay(20); // 50 Hz update rate
}
```

The servos are driven off the powerbank and the laser is driven directly from an Arduino pin since it only draws 25mA peak.

Initial Setup:



Thresholding testing:



Camera:



Videos:

<https://www.youtube.com/watch?v=p8AYsu3IFxU>

Video of robot receiving input from simulation.

With the logitech C920, we were unable to set up adequate thresholding / image processing to get the orange marker in frame. Above a certain t-value, due to the lighting conditions and concrete floor it would start picking up random floor areas in our processed image. But the hardware setup serves as a proof of concept; the serial communication works and robot works as intended.

In the below video, the power bank is off-center and this is apparent in its motion biasing towards the powerbank heavy side but this video also shows the laser go off with simulation (@00:30):

https://www.youtube.com/watch?v=_KvUMkgJjPs